arXiv:2402.11550v2 [cs.CL] 13 Mar 2024

LONGAGENT: Scaling Language Models to 128k Context through Multi-Agent Collaboration

Jun Zhao[1*†], **Can Zu**[1*], **Hao Xu**[1], **Yi Lu**[1], **Wei He**[1], **Yiwen Ding**[1], **Tao Gui**[2†], **Qi Zhang**[1†], **Xuanjing Huang**[1]
[1]School of Computer Science, Fudan University
[2]Institute of Modern Languages and Linguistics, Fudan University
{zhaoj19,qz,tgui}@fudan.edu.cn,czu22@m.fudan.edu.cn

Abstract

Large language models (LLMs) have demonstrated impressive performance in understanding language and executing complex reasoning tasks. However, LLMs with long context windows have been notorious for their expensive training costs and high inference latency. Even the most advanced models such as GPT-4 and Claude2 often make mistakes when processing inputs of over $100k$ tokens, a phenomenon also known as *lost in the middle*. In this paper, we propose LONGAGENT, a method based on multi-agent collaboration, which scales LLMs (e.g., LLaMA) to a context of 128K and demonstrates potential superiority in long-text processing compared to GPT-4. In LONGAGENT, a leader is responsible for understanding user intent and directing team members to acquire information from documents. Due to members' hallucinations, it is non-trivial for a leader to obtain accurate information from the responses of dozens to hundreds of members. To address this, we develop an *inter-member communication* mechanism to resolve response conflicts caused by hallucinations through information sharing. Our experimental results indicate that LONGAGENT offers a promising alternative for long-text processing. The agent team instantiated with LLaMA-7B achieves significant improvements in tasks such as 128k-long text retrieval, multi-hop question answering, compared to GPT-4.

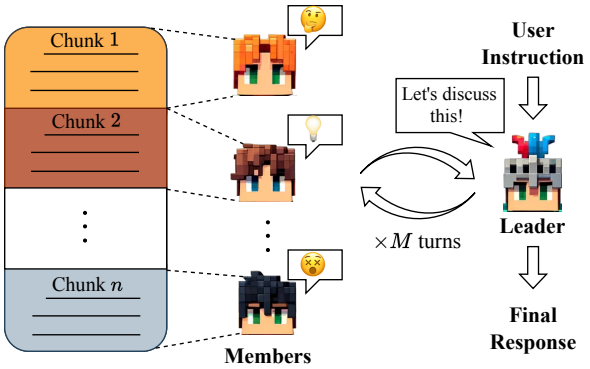

Figure 1: LONGAGENT collaboration scheme. The input long text (left) is segmented into several chunks and assigned to corresponding members. The Leader receives user instruction (right), breaks them down into the simplest sub-problems, convenes members for discussion, ultimately obtaining answers to all sub-problems, and reasons to make the final response.

1 Introduction

Benefiting from increased model scales and massive pre-training corpus, large language models (LLMs) like GPT-4 (OpenAI, 2023) and LLaMA (Touvron et al., 2023) have continuously improved their capabilities. However, due to the inherent quadratic complexity of attention mechanisms, LLMs are often pre-trained with a limited context window to avoid unbearable computational costs. Once the input text length exceeds this limit, LLMs suffer from severe performance degradation (Xiao et al., 2023; Peng et al., 2023). This significantly constrains the effectiveness of LLMs in many practical applications, such as querying information from books, analyzing legal documents, or scientific papers.

Recently, many efforts have been dedicated to addressing the challenges of extending the context window of pre-trained LLMs. The first category of methods considers positional encoding as a crucial aspect to tackle this issue (Press et al., 2022; Chen et al., 2023c; Peng et al., 2023; Chen et al., 2023a). By extrapolation or interpolation, these methods enable positional encoding to adapt to positions "unseen" during the pre-training stage. However, this adaptation process may impair the performance on short sequences acquired during pre-training (Jin et al., 2024). Additionally, as the window size increases, positional biases may decrease the effectiveness of attention mechanisms

*Equal Contributions.
†Corresponding authors.

(Liu et al., 2023), a phenomenon referred to as *lost in the middle*. The second category of methods employs intricate mechanisms such as designing recurrent structures (Zhou et al., 2023; Zhang et al., 2024), token selection (Mohtashami and Jaggi, 2023; Tworkowski et al., 2023), or sliding windows (Xiao et al., 2023; Han et al., 2023), enabling limited context windows to handle longer input texts. While these methods offer advantages in computational efficiency, valuable long-term dependencies may gradually be overlooked with multiple recurrent iterations or window sliding.

In this work, we introduce a promising novel method, termed LONGAGENT, to address the challenge of handling long texts. LONGAGENT achieves processing of documents exceeding $100k$ through multi-agent collaboration. As illustrated in Figure 1, our agent team consists of a leader and multiple members. The leader is responsible for: 1) understanding user intent and organizing discussions among members; 2) supervising communication among members to resolve conflicting opinions; 3) gathering relevant information and reasoning answers. Each member is tasked with responding to the leader's instructions based on the content in the assigned text chunk. Due to members' hallucinations, it is non-trivial for a leader to obtain accurate information from the responses of dozens to hundreds of members. We address this issue through an inter-member communication mechanism. The leader detects members with conflicting opinions during discussions and allows them to interact directly to eliminate hallucinatory responses. In order to comprehensively evaluate LLMs' long-text capabilities, we further extend *Needle in a Haystack*, a recently popular pressure test long-context LLMs. We change the simple fact retrieval to more challenging question-answering that may involve multiple documents. The entities related to answers in the documents have been modified to prevent models from taking shortcuts by relying on internal knowledge. We named the new test *Needle in a Haystack PLUS*[*].

The main contributions of our work are as follows: 1) we propose LONGAGENT, scaling LLMs with $4k$ context size to effectively handle long texts exceeding $100k$ tokens; 2) we construct a larger benchmark, *Needle in the Haystack PLUS*, enabling more comprehensive evaluation on LLMs' long-text capabilities. 3) experimental results show that LONGAGENT, built upon LLaMA-7B, exhibits potential surpassing GPT-4 in long text processing. This improvement strongly suggests that multi-agent collaboration is a promising direction for improving long-text processing.

[*]https://github.com/zuucan/NeedleInAHaystack-PLUS

2 LONGAGENT for Long Text Processing

2.1 Method Overview

As illustrated in Figure 2, we use long-text question answering as an example to elucidate the working mechanism of LONGAGENT. Given a long text x and a user query q, LONGAGENT searches for relevant evidence from the long text x and reasons for the final response r through collaborative efforts of multiple agents. This process involves the following 4 steps:

(1) **Selecting Members**. LONGAGENT, as a task-agnostic method for long-text processing, supports constructing customized agent teams based on the task to be processed. For example, given the task description d ="*Answer the question based on the given passages*", the leader selects the QA expert model to instantiate team members for more accurate and reliable responses. Then, we partition the long text x into n chunks $\{c_1, c_2, ..., c_n\}$ of predefined size and distribute them accordingly to n members $\{m_1, m_2, ..., m_n\}$ for processing.

(2) **Collaborative Reasoning**. For a complex user query q, the leader systematically breaks q down into multiple sub-questions and organizes members to collaborate in searching for clues from their respective chunks. As shown in fig 2, to answer q = "*Which team does the player named 2015 Diamond Head Classic's MVP play for?*", the leader first identifies who won the MVP of the 2015 Diamond Head Classic, and then further identifies which team this player play for. For more complex queries, collaborative reasoning will involve multiple rounds.

(3) **Resolving Conflict**. Due to model hallucinations, some members may respond with false answers not mentioned in the document. Obviously, these false answers conflict with the correct one. The leader needs to identify such conflicts and ascertain the correct answer explicitly mentioned in the document.

(4) **Deducing the Answer**. When the leader believes that the current discussion history is sufficient to derive the answer to the user query, it ends the discussion and provide the final response.

In the following sections, we will elaborate on

the details of the aforementioned four steps.

2.2 Selecting Experts to Instantiate Members

The working mechanism of LONGAGENT involves coordinating members and integrating their processing results of respective chunks to form the final response. Different long-text tasks require members to possess diverse text processing capabilities. To address this challenge, we utilize expert models to construct task-specific agent teams, aiming to generate more accurate responses.

Construction of expert models: This step aims to build a candidate set of expert models $E = \{e_1, e_2, ..., e_s\}$, where different experts excel in different tasks. For strong models (e.g., GPT-4, GPT-3.5), we employ a prompt-based approach to construct expert models. Specific prompt templates are used to elicit the internal capabilities of the model for the corresponding tasks. For example, "*You are an expert in answering questions, adept at searching for relevant information from given documents and providing answers*." The benefit of this approach is that all expert models can share the same parameters. For weak models (e.g., LLaMA-7b), we utilize a fine-tuning-based approach to inject task knowledge to make them experts in the corresponding tasks. The advantage of this method is that it can produce more accurate responses for specific tasks.

Member selection: Given a natural language description d of a task to be processed, we prompt the leader to select a suitable expert $e_i \in E$ to play roles as team members. We assume that the task at hand requires only one particular expert to complete. For complex tasks that require collaboration among experts with different skill sets, we leave it as future work. The prompt template is illustrated in tab. 2. All members in the team share the parameters of the expert model e_i.

2.3 Collaborative Reasoning

To respond to user query q, the leader needs to coordinate members to process text and acquire relevant information. As the core of the team, the leader continuously executes the following decision-making process:

$$a \sim \text{Leader}(a|S, q), \tag{1}$$

where q denotes the user query, $S = \{s_1, s_2, ..., s_m\}$ represents the historical dialogue states, and s_i represents a round of dialogue composed of an instruction from the leader and responses from all members. The leader sample an action $a \in \{\texttt{NEW_STATE}, \texttt{CONFLICT}, \texttt{ANSWER}\}$ based on the dialogue history S and the user query q. If $a = \texttt{NEW_STATE}$, it it indicates that the information contained in the preceding i rounds of dialogue history is insufficient to respond to query q. Therefore, the leader initiates the next round of dialogue s_{i+1}, and generates new instructions to further gather information. Conversely, if $a = \texttt{ANSWER}$, it signifies that the leader deems the currently collected information sufficient to derive the final answer, and the collaborative process terminates accordingly. CONFLICT is a special state other than the two aforementioned states, indicating that the leader perceives conflicting answers from the members in the current round of dialogue s_i. We elaborate on this situation in the next subsection.

2.4 Resolving Conflicts

Due to model hallucinations, members may respond with content not mentioned in their chunks. The dialogue in Step 2 of Figure 2 serves as an example, where two members respectively believe *Buddy Hield* and *Mark Gibson* to be the MVP of the 2015 Diamond Head Classic, despite the latter not being mentioned in the text chunk. We address this issue through *inter-member communication*, inspired by the following empirical findings: 1) When there is answer to the leader's instruction in the chunk, the member often provides correct responses rather than generating hallucinations; 2) When there are no answers in the chunk, the model frequently fabricates an answer instead of responding with 'no mention,' even after supervised fine-tuning. Using this feature, the leader first identifies the member IDs where answers conflict and then requests these members to share chunks pairwise and provide answers again:

$$\text{hallucination} = m_i(c_i), \tag{2}$$

$$\text{Truth} = m_j(c_j), \tag{3}$$

$$\text{Truth} = m_j(c_j \oplus c_i) \tag{4}$$

Here, c_i and c_j respectively represent two text chunks, where c_j contains the correct answer while c_i does not. m_i and m_j denote two members. Our experimental results demonstrate that sharing text chunks is a simple yet effective strategy. The majority of members experiencing hallucination tend

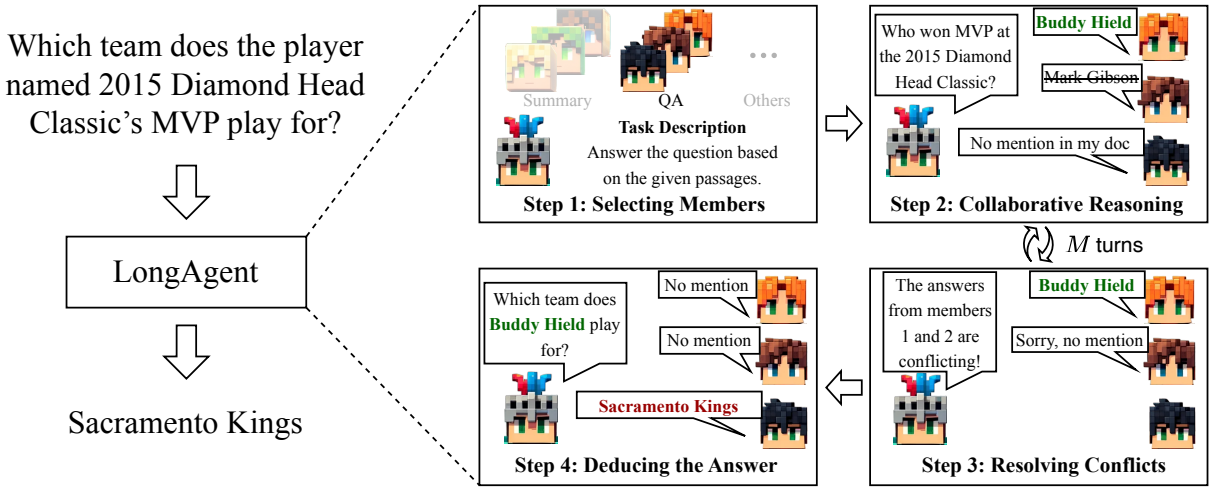

Figure 2: An Overview of the LongAgent. In step 1, the leader constructs a customized agent team based on the description of the task to be handled. In the second and third steps, the leader organizes the team to gather information from documents and resolve conflicts. This process may continue for multiple rounds until the leader deems enough information has been gathered to generate the final response, which is then exported in the step 4.

to correct their original responses upon receiving the chunk containing the correct answers, resulting in accurate output. While we acknowledge some advanced mechanisms for mitigating hallucination issues, such as multi-agent debate (Du et al., 2023) and reflection (Shinn et al., 2023), these are not the focus of this paper; we leave them as avenues for future research.

3 Experimental Setup

3.1 Evaluation Protocol

Needle-in-a-Haystack PLUS: The *Needle-in-a-Haystack* (Kamradt, 2023) is currently one of the most popular testbed for evaluating the capability to handle long texts. In this setup, a fact or statement of interest (the *needle*) is placed within a lengthy distracting document (the *haystack*), and the model is tasked with retrieving this hidden key information. Performance is evaluated by varying the position of the needle within the distracting document and the length of the distracting document itself. To assess the long-text capabilities more comprehensively, we propose *Needle-in-a-Haystack PLUS*, which shifts the focus from simple fact retrieval to more challenging single-document/multi-document question answering tasks. In *Needle-in-a-Haystack PLUS*, the *needle* represents the document(s) containing the answers, while the *haystack* comprises distracting documents. The model must locate one or more relevant documents scattered within the haystack and reason the correct answers from them. For the purpose of evaluation, we deliberately select questions with definite answers as test data, such as questions where the answer is a specific entity or a simple yes/no response. To mitigate the risk of models relying on internal knowledge to answer, we replace entities directly related to the answer within the documents with fictional entities. In Appendix A, we elaborate on the collecting process of the test data, including single-document QA and multi-hop QA involving multiple documents.

Synthetic Tasks: In addition to the *Needle-in-a-Haystack PLUS* test, we also selected two widely used long sequence evaluation tasks (Mohtashami and Jaggi, 2023; Liu et al., 2023; Zhang et al., 2023): long-text retrieval and numerical comparison. We choose them for the following reasons: (1) Similar to the needle-in-a-haystack task, these synthetic tasks all use ACC as the evaluation metric, facilitating evaluation. The difference in metrics can directly reflect the difference in the model's long sequence processing capabilities. (2) We can automatically synthesize training data for fine-tuning open-source models. The long-text retrieval task includes the following three subtasks: 1) PassKey Retrieval: Retrieving hidden keys in a noisy long context; 2) Number Retrieval: Locating repeated hidden numbers in a noisy long context. 3) KV Retrieval: Finding the corresponding value from a dictionary and a key. Numerical comparison requires the model to find numbers that meet specific requirements from a numeric string of magnitude $100k$ tokens, such as the top K numbers,

median, etc., where K can be 1, 2, or 3.

3.2 Compared Methods

PI (Chen et al., 2023c). Extending the context window sizes of RoPE-based pretrained large language models by position interpolation.

YARN (Peng et al., 2023). YaRN is an improved method to efficiently extend the context window. This work directly modifies the PE to expand to a theoretically infinite context length.

Claude2.1 (Anthropic, 2023). The Claude 2.1 released by Anthropic Corporation features a context window of 200K tokens and has significantly reductions in rates of model hallucination.

GPT-4 Turbo (OpenAI, 2023). The GPT-4 Turbo model from OpenAI offers a context window of 128K and can process text exceeding 300 pages within a single prompt.

3.3 Implementation Details

To build an agent team, we perform supervised fine-tuning on LLaMA2-7b-base. Within the agent team, the Leader is responsible for coordinating Members to accomplish various tasks. We utilize GPT-4 to generate $1,000$ interaction trajectories for each task to train the Leader, and manually verified the correctness of these interaction trajectories. Members are tasked with processing documents based on the Leader's instructions. To achieve this, we train QA experts, retrieval experts, and mathematical experts for instantiating members. Regardless of the number of members instantiated, they all share the parameters of a single expert model. Training data for QA experts are sourced from the SQuAD training set, consisting of $25,000$ samples. Among these, $10,000$ samples contain answers within the documents, while the remaining $15,000$ samples do not, requiring the model to abstain from answering. We extended document lengths to 2500-3000 tokens through concatenation. Training data for retrieval experts and mathematical experts are synthesized automatically, with $10,000$ documents generated for each task, ranging in length from $1k$ to $3k$ tokens, and information to be retrieved evenly placed at random positions within the documents. It's important to note that all training data is non-overlapping with the final evaluation data. Please refer to Appendix B for prompts and interaction trajectories for all tasks.

Methods	Retrieval			Numerical Comparison
	PassKey	Number	KV	
GPT-4	**1.000**	**1.000**	0.890	0.600
Kimi-Chat	0.981	0.954	0.536	0.126
Claude2.1	0.978	0.981	0.654	0.323
YaRN	0.927	0.566	–	0.171
Ours-GPT3.5	**1.000**	**1.000**	0.638	0.511
Ours-LLaMA-7B	**1.000**	**1.000**	**0.966**	**0.625**

Table 1: The experimental results (accuracy) on four synthesis tasks.

4 Results and Discussion

4.1 Overall Performance

To demonstrate the superiority of LongAgent in handling long texts, we compare it against powerful commercial models GPT-4 Turbo and Claude 2.1, as well as the state-of-the-art academic methods for long-text processing, PI and YARN.

Through multi-agent collaboration, fine-tuning LLaMA with only a 4k context window effectively handles contexts of up to 128k.

The results for the *Needle-in-a-Haystack PLUS* are shown in Figure 3 and 4, respectively. LONGAGENT, constructed from fine-tuned LLaMA2-7B, significantly outperforms GPT-4 across document length ranging from 1k to 128k, with an average improvement of 19.53% (from 62.00% to 81.53%) under the single-document setting, and an average improvement of 4.96% (from 50.37% to 55.33%) under the multi-document setting. Considering that LONGAGENT is fine-tuned on downstream tasks, for fair comparison, we fine-tune PI and YARN on task data with lengths ranging from 1 to $16k$ (training with longer contexts exceeds our hardware limitations). Experimental results demonstrate that when the length of the test document exceeds the maximum length trained on, PI and YARN fail to generate results properly. Even within the 0-$16k$ range (corresponding to the first two columns of the grid), the average performance of LONGAGENT surpasses that of PI and YARN. The results on the four synthetic tasks are shown in Table 1. From the table, we can observe that LONGAGENT supported by fine-tuned LLaMA2-7B model outperforms all baseline models, achieving or approaching 100% accuracy on the three retrieval-type tasks. This demonstrates the superiority of LONGAGENT in handling various long-text tasks.

For LONGAGENT supported by more powerful models like GPT-3.5, fine-tuning is not neces-

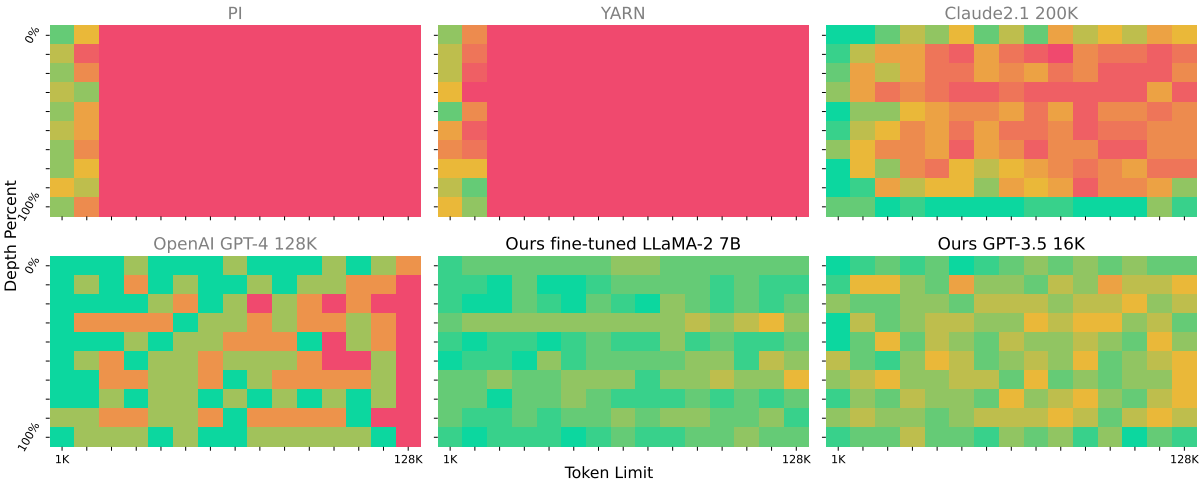

Figure 3: The Comparison of Results of *Needle-in-a-Haystack PLUS* in Single-Document Question Answering Setting. Under the LANGAGENT scheme, our fine-tuned LLaMA2-7B model achieved an average accuracy improvement of 19.53% compared to GPT-4 across the range from 1k to 128k (increasing from 62.00% to 81.53%).

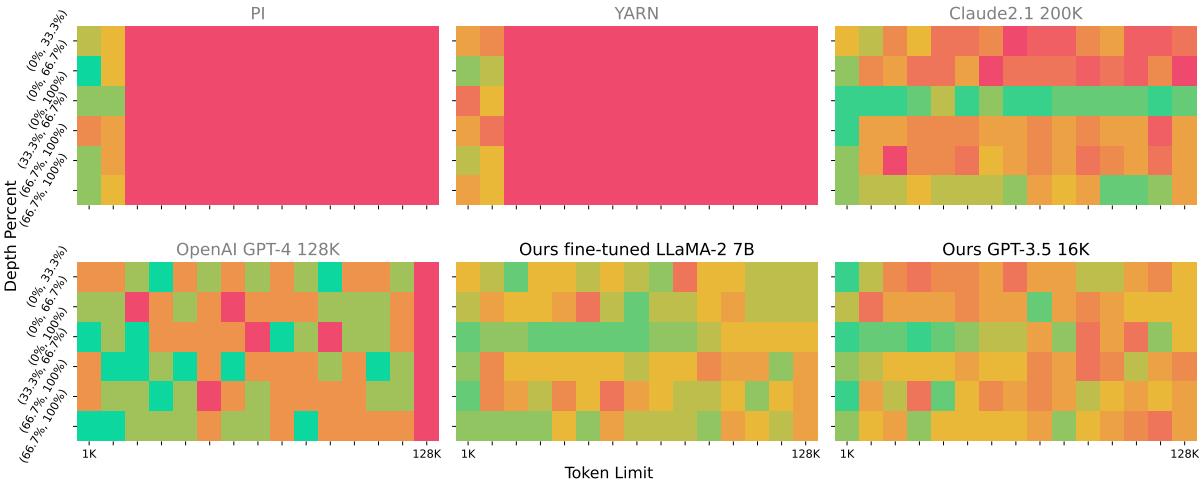

Figure 4: The Comparison of Results of *Needle-in-a-Haystack PLUS* in Multi-Document Question Answering Setting. Under the LangAgent scheme, our fine-tuned LLaMA2-7B model achieved an average accuracy improvement of 4.96% compared to GPT-4 across the range from $1k$ to $128k$ (increasing from 50.37% to 55.33%).

sary.

Through prompting, GPT-3.5 can simultaneously act as a leader and members with specific skills. Despite having only a $16k$ context window, we found that the LONGAGENT supported by GPT-3.5 can effectively handle documents far exceeding $16k$ in length. Specifically, in the *needle-in-a-haystack PLUS* task, LONGAGENT achieved improvements of 6.780% and 1.5% over GPT-4 in single-doc and multi-doc settings, respectively. For the four synthetic tasks in Table 1, LONGAGENT also achieved perfect scores in two retrieval tasks with $100k$ length documents. For KV retrieval and numerical comparison tasks, it also outperformed the majority of baselines.

Although we only tested inputs ranging from 1k to 128k, LONGAGENT demonstrates potential in handling inputs exceeding 128k in length.

In Figure 3 and 4, we observed the *lost in the middle* phenomenon with Claude 2.1. Specifically, as the input length increases, Claude2.1's average accuracy gradually decreases. However, in the first and last rows of the Claude subfigure in Figure 3, and the third row of the Claude subfigure in Figure 4, relatively high accuracy is consistently maintained. This suggests that Claude2.1 can effectively model the beginning and end of long texts but fails to utilize key information in the middle effectively. LONGAGENT avoids direct processing of long texts through chunking. Regardless of the input length, the chunk size remains constant, thus avoiding the 'lost in the

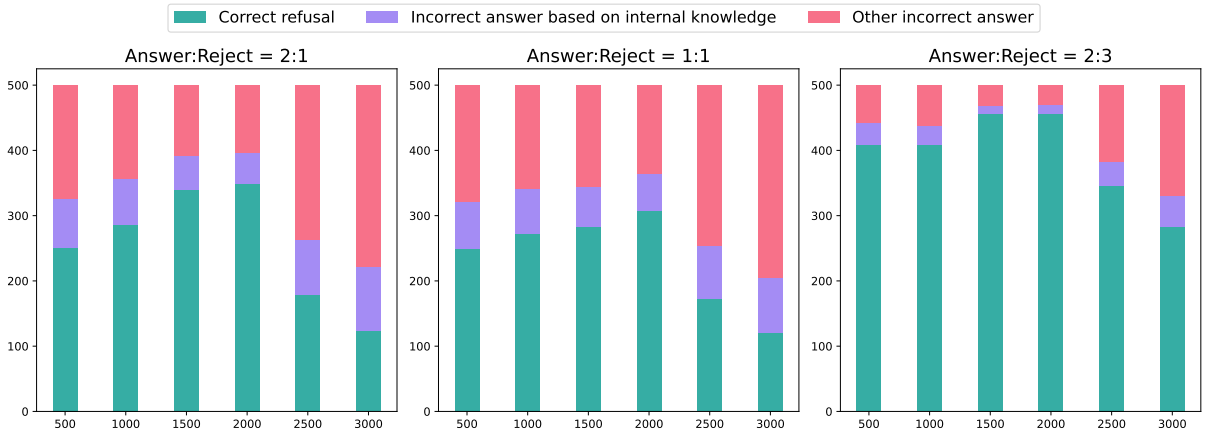

Figure 5: The influence of data recipe on model hallucinations. 'Answer' and 'Reject' represent two types of data. For the former, the documents contain answers to questions; whereas for the latter, they do not.

middle' phenomenon. Although longer inputs may complicate agent interactions, experimental results show no significant performance decrease for LONGAGENT. Overall, LONGAGENT has the potential to handle inputs exceeding $128k$ in length.

4.2 Hallucination Analysis

We found that the errors of LongAgent are mainly due to a type of hallucination problem: when the chunk of a member does not contain information related to the instruction of the Leader, the member sometimes answers based on internal knowledge or fabricates a wrong answer as a response. In this subsection, we explore the impact of two key factors, the recipe of training data and chunk size, on model hallucination. As shown in Figure 5, with the increase of 'Reject' type data in the training data, the proportion of the model correctly refusing to answer increased from 51.0% to 78.6%. However, the increase of 'Reject' data also slightly impairs the model's ability to answer questions. As shown in Figure 8, when the ratio of 'Answer:Reject' increases from 2:1 to 2:3, the accuracy of the model decreases from 83.3% to 78.3%, and there are also a small number of cases where the document contains the answer but refuses to answer.

In addition to the data proportion, chunk size is also an important factor affecting model hallucination. As shown in Figure 5, when the chunk size increases from 500 to $2,000$, the hallucination problem is alleviated. This is mainly because the length of our training data is about $3,000$ tokens, and increasing the chunk size reduces the gap with the length of the training

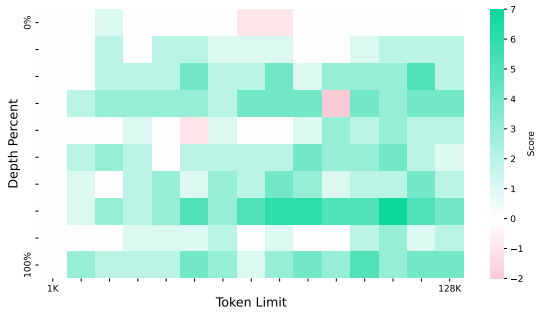

Figure 6: Improved accuracy through *inter-member communication* mechanism.

data. However, when the chunk size exceeds $2,000$, further increasing the chunk size significantly exacerbates model hallucination. We speculate that this is because when the sequence length is too long, the model's inadequacy in document modeling becomes more prominent. Therefore, we believe that while researching how to construct larger context windows, we should not neglect the modeling of text within a $4k$ window.

4.3 Ablation Study

In Section 2.4, we address conflicts between members through *inter-member communication*. To demonstrate the effectiveness of this mechanism, we calculate the difference in model accuracy before and after introducing this mechanism. As shown in Figure 6, the *inter-member communication* mechanism leads to an average accuracy improvement of 18.9% across a range of input text lengths from $1k$ to $128k$. Furthermore, the number of members increases with the length of the text, and the number of members experiencing hallucinations also grows. In this context, the

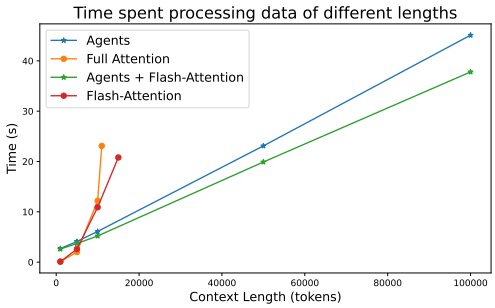

Figure 7: LONGAGENT scheme exhibits significantly superior time and memory efficiency compared to directly perform full attention on long texts.

improvement in accuracy brought about by conflict resolution becomes even more evident.

4.4 Efficiency Advantage

Thanks to chunking of long texts, LONGAGENT's time complexity for processing long texts is $\mathcal{O}(N)$. In this subsection, we empirically verify this point. As shown in Figure 7, the latency of LONGAGENT within the range of $1k$-$100k$ almost grows linearly with length. For Full Attention, which has quadratic complexity, the inference latency increases rapidly regardless of the use of techniques such as flash attention. The latency of Full Attention when processing 10k tokens has already exceeded that of LONGAGENT processing $50k$ tokens. Furthermore, without specific memory optimization techniques, a single A100 GPU with 80G memory can only support text inference up to 11k in length, and even with flash attention, this number can only be increased to $15k$. Under the same settings, LONGAGENT can process contexts of around $100k$ with less than 40G of memory.

5 Related Works

5.1 Long-text Modeling

Several methods have been proposed to extend the positional encoding (PE) for handling longer sequences. Initially, approaches like RoPE and PI (Chen et al., 2023c) attempted to interpolate position indices within pre-trained limits, but neglected frequency variations. Recent advancements include "NTK-aware" (Bloc97, 2023a) interpolation and "Dynamic NTK" (Bloc97, 2023b) interpolation, which address high-frequency component losses. Additionally, "NTK-by-parts" (Bloc97, 2023c) interpolation outperforms others when fine-tuned on longer-context data. Another popular approach for managing longer sequences involves constraining global causal attention to local attention. ReRoPE (Su, 2023) truncates context lengths during pretraining and LM-Infinite (Han et al., 2023) restricts attention to a chevron-shaped window. Mohtashami and Jaggi (2023) insert landmark tokens after text fragments, while Zhang et al. (2024) propose beacon tokens for summarizing fragments. In contrast, our method effectively circumvents the risk of losing valuable contextual information while utilizing only a small amount (hundreds of agent interaction tracks) for fine-tuning, thereby reducing training costs.

5.2 LLM-Based Multi-Agent Systems

In recent years, LLM-based multi-agent systems have garnered widespread attention in academia. Numerous efforts have been dedicated to leveraging cooperation among individuals to enhance the efficiency and accomplish more complex reasoning tasks (Du et al., 2023; Wang et al., 2024; Akata et al., 2023; Hao et al., 2023). To enable agents to effectively address a variety of dynamic tasks in the real world, researchers have also integrated external tools into the agents' decision-making processes (Cai et al., 2023; Gao et al., 2023; Paranjape et al., 2023), enabling them to perform accurate computations and retrieve the latest information from databases or search engines. In these approaches, the most relevant ones to ours are PEARL (Sun et al., 2023) and MemWalker (Chen et al., 2023b). PEARL enhances the model's focus on relevant content within long texts by calling self-generated pseudo APIs. However, it can only handle long texts within the agent's context window and is ineffective for longer texts. Although MemWalker enables agents to process longer texts through a tree-based summarization approach, crucial information may be lost after multiple summarizations, causing the agent to get lost in irrelevant contexts.

6 Conclusions

This paper proposes LONGAGENT, a novel long-text processing approach based on multi-agent collaboration. LONGAGENT scaling LLMs with $4k$ context size to effectively hadle long texts exceeding $100k$ tokens. The proposed *inter-member communication* mechanism alleviates the member hallucination when they reading documents, thus facilitating effective management by the leader of

dozens to hundreds of members. We have also developed *Needle-in-a-Haystack Plus* to facilitate a comprehensive assessment of the LLM's capability with long texts. Our experimental results indicate that LONGAGENT offers a promising alternative for long-text processing.

Limitations

LONGAGENT still has some drawbacks. Unlike general SFT data that only provides a prompt and a final response, LONGAGENT's training data consists of interaction trajectories of multiple agents. Therefore, the construction cost of a single data point is higher, especially for tasks with more complex interaction trajectories. In addition, as the core of the agent squad, the Leader needs to make reasonable decompositions of the original complex problem and recruit members to solve it, which places higher demands on the Leader's reasoning and generalization abilities. For example, in the 'needle in a haystack' experiment, LONGAGENT improved by 19.53% in a single-document setting compared to GPT-4, but this number dropped to 4.96% when switching to a more complex multi-document setting. The main reason is that the reasoning ability of the LLaMA2-7B model is not sufficient to accurately decompose some complex problems.

References

Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, and Eric Schulz. 2023. Playing repeated games with large language models.

Anthropic. 2023. Model card and evaluations for claude models. Website. https://www.anthropic.com/product.

2023a Bloc97. 2023a. Ntk-aware scaled rope allows llama models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation. https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have/.

2023b Bloc97. 2023b. Dynamically scaled rope further increases performance of long context llama with zero fine-tuning. https://https://www.reddit.com/r/LocalLLaMA/comments/14mrgpr/dynamically_scaled_rope_further_increases//.

2023c Bloc97. 2023c. Ntk-aware interpolation "by parts" correction, 2023. URL https://github.com/jquesnelle/scaled-rope/pull/1.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large language models as tool makers.

Guanzheng Chen, Xin Li, Zaiqiao Meng, Shangsong Liang, and Lidong Bing. 2023a. Clex: Continuous length extrapolation for large language models.

Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. 2023b. Walking down the memory maze: Beyond context limit through interactive reading.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023c. Extending context window of large language models via positional interpolation.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate.

Difei Gao, Lei Ji, Luowei Zhou, Kevin Qinghong Lin, Joya Chen, Zihan Fan, and Mike Zheng Shou. 2023. Assistgpt: A general multi-modal assistant that can plan, execute, inspect, and learn.

Chi Han, Qifan Wang, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. 2023. Lm-infinite: Simple on-the-fly length generalization for large language models.

Rui Hao, Linmei Hu, Weijian Qi, Qingliu Wu, Yirui Zhang, and Liqiang Nie. 2023. Chatllm network: More brains, more intelligence.

Hongye Jin, Xiaotian Han, Jingfeng Yang, Zhimeng Jiang, Zirui Liu, Chia-Yuan Chang, Huiyuan Chen, and Xia Hu. 2024. Llm maybe longlm: Self-extend llm context window without tuning.

Greg Kamradt. 2023. Needle in a haystack - pressure testing llms. Website. https://github.com/gkamradt/LLMTest_NeedleInAHaystack.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts.

Amirkeivan Mohtashami and Martin Jaggi. 2023. Landmark attention: Random-access infinite context length for transformers.

OpenAI. 2023. Gpt-4 technical report.

Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. 2023. Art: Automatic multi-step reasoning and tool-use for large language models.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. Yarn: Efficient context window extension of large language models.

Ofir Press, Noah A. Smith, and Mike Lewis. 2022. Train short, test long: Attention with linear biases enables input length extrapolation.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning.

Jianlin Su. 2023. Rectified rotary position embeddings. https://github.com/bojone/rerope.

Simeng Sun, Yang Liu, Shuohang Wang, Chenguang Zhu, and Mohit Iyyer. 2023. Pearl: Prompting large language models to plan and execute actions over long documents.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, and Xavier Martinet. 2023. Llama: Open and efficient foundation language models.

Szymon Tworkowski, Konrad Staniszewski, Mikołaj Pacek, Yuhuai Wu, Henryk Michalewski, and Piotr Miłoś. 2023. Focused transformer: Contrastive training for context scaling.

Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. 2024. Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering.

Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. 2024. Soaring from 4k to 400k: Extending llm's context with activation beacon.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Qihao Wu, Junhao Chen, Zihang Xu, Zhenning Dai, Xu Han, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2023. Infinitebench: 128k long-context benchmark for language models.

Wangchunshu Zhou, Yuchen Eleanor Jiang, Peng Cui, Tiannan Wang, Zhenxin Xiao, Yifan Hou, Ryan Cotterell, and Mrinmaya Sachan. 2023. Recurrentgpt: Interactive generation of (arbitrarily) long text.

A Collecting Needle-in-a-Haystack PLUS Testing Data

The test data consists of two parts: single-document QA and multi-document QA. Below, we will elaborate on the construction process of each.

A.1 Single-document QA

For single-document QA, the test data is constructed based on SQuAD (Rajpurkar et al., 2016). SQuAD is a large-scale machine reading comprehension dataset containing one hundred thousand questions, selected from over 500 Wikipedia documents. Each question in the dataset has its answer as a text segment extracted from the given document. We randomly select 100 samples from the training set of SQuAD and replace the key entities in them. These 100 samples' documents are used as *needles*, while documents from other samples in the training set are randomly selected as *haystacks*, which provide distractor information. The length of *haystack* ranges from $1,000$ to $128,000$ tokens with equal intervals, totaling 15 different lengths. The depth of the needle ranges from 0% to 100% with equal intervals, totaling 10 different depths. Depth indicates the position of the needle in the *haystack*. A depth of 0% means the *needle* is at the beginning of the haystack, while 100% indicates it's at the end. For each length and depth, we randomly select 10 *needles* to construct 10 test samples.

A.2 Multi-document QA

For multi-document QA, questions require information from two or more documents to reason the final answer. We construct test samples based on HotpotQA (Yang et al., 2018), a widely adopted multi-document QA dataset. We select 60 questions from the validation set of HotpotQA that require information from two documents to answer. Each sample contains two *needles*, and the haystack is still composed of distractor documents. The length of *haystack* ranges from $1,000$ to $128,000$ tokens with equal intervals, totaling 15 different lengths. The two *needles* are randomly scattered at the depth of 0%, 33%, 66%, and 100% of the haystack, resulting in 6 combinations: $(0\%, 33\%)$, $(0\%, 66\%)$, $(0\%, 100\%)$, $(33\%, 66\%)$, $(33\%, 100\%)$, and $(66\%, 100\%)$. For specific lengths and needle positions, we randomly select 10 *needles* to construct 10 test samples for evaluation.

B Trajectory for Each Task

B.1 Single-document Question Answering

Dataset: Squad

Leader first input

First, the leader recruits members according to the task description as shown in Table 2.

You need to recruit a team of members to solve a task. Select the appropriate member based on the task description.

 # Task Description:

{task_description}

 # Members List:

QA member: Good at solving Question Answering problems.

KV member: Good at finding the corresponding value from a dictionary.

NS member: Good at locating repeated hidden numbers in a noisy long context.

PassKey member: Good at retrieving hidden keys in a noisy long context.

Math member: Good at finding special integers in a lengthy list.

Your output must following the JSON format: {{"type": "member", "content": "your_chosen_member"}}

Table 2: Prompt template for Leader first input on the Single-document Question Answering Task. The content of #Task Description is derived from user input.

Leader first output

{"type": "member", "content": "QA member"}

Leader next input

After recruiting members, the leader gives specific instruction as shown in Table 3.

Member first input

The prompt for the Member first input is shown in Tabel 4. The content of #Document is a part of the complete context, with variations for each member. The content of #Instruction originates from the first output of the leader.

Member first output

The leader will exclude members who refuse to answer and group together members who provide the same answer.

You are the leader of a team of {member_nums} members. Your team will need to collaborate to solve a task. The rule is:

1. Only you know the task description and task objective; the other members do not.
2. But they will receive different documents that may contain answers, and you need to send them an instruction to query their document.
3. Your instruction need to include your understanding of the task and what you need them to focus on. If necessary, your instructions can explicitly include the task objective.
4. Finally, you need to complete the task based on the query results they return.

 # Task Description:

Answer the question based on the given passages. The answer must be extracted from the given passages.

 # Task Objective:

In which publication did Sander publish an article questioning racial preferences in law schools?

 # Generate Instruction for Members:

Now, you need to generate an instruction for all team members. You can ask them to answer a certain question, or to extract information related to the task, based on their respective documents.

Your output must following the JSON format: {{"type": "instruction", "content": "your_instruction_content"}}

Table 3: Prompt template for Leader next input on the Single-document Question Answering Task. The content of #Task Description and #Task Objective is derived from user input.

Clusters: {'the Wall Street Journal': [1, 7, 8, 13, 15, 17, 18, 22, 23, 24, 28, ,29 31, 32, 33, 34, 35, 37, 39, 42, 43], 'the Yale Law Journal': [5], 'The Wall Street Journal': [20]}

Member next input

Prompt template for Member next input is the same as Member first input shown in Table 4. The content of #Document is the merge of the document from two members.

Select member 43 randomly from Cluster 1 and member 5 randomly from Cluster 2. First, merge the documents of member 43 and member 5 to obtain the new result.

 # Document:
{member_document}

 # Instruction:
Answer the question based on the given passages. The answer must be extracted from the given passages. Question: In which publication did Sander publish an article questioning racial preferences in law schools?

You are an experienced writer; please summarize the content in the document related to the instructions in a <scratchpad> tag, then describe your response." Your output must following the JSON format: {{"type": "response", "content": "your_response_content"}}
The "content" needs to be as concise as possible.

Table 4: Prompt template for Member first input on the Single-document Question Answering Task.

Member 43 and Member 5 merge: the Yale Law Journal

Cluster 1 is removed because its merged output matches the original output of member 5 but not member 43. Next, randomly select member 5 from Cluster 2 and member 20 from Cluster 3. Merge the documents of these two members to create the new result.

Member 5 and Member 20 merge: Yale Law Journal

Remove Cluster 3 because the merged output is identical to the original output of member 5 and differs from the original output of member 20.
At this point, there is only one cluster left, thus all members have only one unique result. The leader will provide the final answer based on this result.
Leader last input Leader needs to determine whether the final answer can be obtained. If not, further instructions will be provided. In this task, the final answer can be obtained. Details are shown in Table 5.

Leader last output
"type": "answer", "content": "Yale Law Journal"

Here are the responses from all the members. Each member sees different segments of a document, and these segments do not intersect with each other. The correct answer may appear in any one or several members' responses.
Note that if a minority of members find information relevant to the question while the majority reply that the document does not contain information relevant to the question, you should pay attention to the replies from those members who found relevant information.
 # Member Response:
Member 1: Yale Law Journal

 # Task Description:
Answer the question based on the given passages. The answer must be extracted from the given passages.

 # Task Objective:
In which publication did Sander publish an article questioning racial preferences in law schools?

 # Determination:
Based on the above information, you need to determine if you can solve the task objective. You have two choices:
1. If members' responses cannot solve the task objective, or if their responses contain conflicting answers, provide a new instruction for them to answer again.
2. Else, if the task objective can be solved, give your final answer as concisely as you can, using a single phrase if possible. Do not provide any explanation.
Your output must following the JSON format: {{"type": "answer", "content": "your_answer_content"}} or {{"type": "instruction", "content": "your_instruction_content"]}}

Table 5: Prompt template for Leader last input on the Single-document Question Answering Task.

B.2 Multi-document Question Answering

Dataset: Hotpotqa
Leader first input
First, the leader recruits members according to the task description as shown in Table 2.
Leader first output
{"type": "member", "content": "QA member"}

Leader second input
For multi-document Question Answering, the questions require answers that integrate the content of multiple documents. The details are shown in Table 6.
Leader second output
The leader needs to break down the original question, so the leader's second output is the first sub-question.
"type": "instruction", "content": "What was the population of Leshan in 2002?"
Member first input
The prompt for the Member first input is shown in Tabel 7. The content of #Document is a part of the complete context, with variations for each member. The content of #Instruction originates from the first output of the leader.

Member first output
Member 1: 121,277
Member 2: The document does not contain the answer.
Member 3: The document does not contain the answer.
. . . . . .
Member 30: The document does not contain the answer.
Member 31: The document does not contain the answer.
Leader third input
The prompt for the Leader third input is shown in Tabel 8. At this moment, Leader still retains the dialogue history from the first round.
Leader second output
{"type": "instruction", "content": "What was the population of Quanzhou in 2002?"}
Member second input
The prompt for the Member second input is shown in Tabel 9. The content of #Document is a part of the complete context, with variations for each member. The content of #Instruction originates from the second output of the leader.
Member second output
Exclude the members who refuse to respond, and cluster the members who provide the same answers.

Clusters: {'1.5 million': [7], '658,798': [11]}

Member third input
The prompt for the Member third input is shown in Tabel 10. The content of #Document is the merge of the document from Member 7 and Member 11.
Member third output

{"type": "response", "content": "658,798"}

At this point, there is only one cluster left, thus all members have only one unique result.
Leader last input
The prompt for the Leader last input is shown in Tabel 11. At this moment, Leader still retains the dialogue history of the previous two rounds.
Leader last output
Leader integrates the answers to the two sub-problems to obtain the final answer to the original problem.

{"type": "answer", "content": "Quanzhou"}

B.3 Retrieve

The Retrieve-type tasks are divided into three types: Retrieve.KV, Retrieve.PassKey, and Retrieve.Number. Although the task descriptions vary for different tasks, the interaction trajectories are similar, and the prompt template is also the same. Therefore, Retrieve.KV task is chosen here as an example for demonstration.
Leader first input
First, the leader recruits members according to the task description as shown in Table 2.

Leader first output
{"type": "member", "content": "KV member"}

Leader next input
The prompt for the Leader next input is shown in Tabel 12.
The content of #Task Description and #Task Objective is derived from user input.

Leader next output
{ "type": "instruction", "content": "Finding the corresponding value from a dictionary and a key. Key: "2b114db0-d87e-42d2-9b4c-0b1f115976ad" The value associated with the specified key is: " }
Member first input
The prompt for the Member first input is shown in Tabel 13.
The content of #Document is a part of the complete context, with variations for each member.
The content of #Instruction originates from the first output of the leader.

You are the leader of a team of {member_nums} members. Your team will need to collaborate to solve a task. The rule is:
1. Only you know the task description and task objective; the other members do not.
2. But they will receive different documents that may contain answers, and you need to send them an instruction to query their document.
3. Your instruction need to include your understanding of the task and what you need them to focus on. If necessary, your instructions can explicitly include the task objective.
4. Finally, you need to complete the task based on the query results they return.

 # Task Description:
Answer the question based on the given passages. Only give me the answer and do not output any other words.

 # Task Objective:
Did Leshan or Quanzhou have a population of 658,798 in 2002?

 # Generate Instruction for Members:
Now, you need to generate an instruction for all team members. You can ask them to answer a certain question, or to extract information related to the task, based on their respective documents.
Your output must following the JSON format: {{"type": "instruction", "content": "your_instruction_content"}}

Table 6: Prompt template for Leader second input on the Multi-document Question Answering Task. The content of #Task Description and #Task Objective is derived from user input.

 # Document:
{member_document}

 # Instruction:
What was the population of Leshan in 2002?

You are an experienced writer; please summarize the content in the document related to the instructions in a <scratchpad> tag, then describe your response." Your output must following the JSON format: {{"type": "response", "content": "your_response_content"}}
The "content" needs to be as concise as possible.

Table 7: Prompt template for Member first input on the Multi-document Question Answering Task.

Member first output
Except for the members who refuse to answer, member 1, member 9, and member 12 provide three different responses.
Clusters:
{'f1ec3dd8-b6e7-4af5-a589-fda0b7693f93': [1], 'd3999999-9999-4999-8999-999999999999': [9], 'd9999999-9999-4999-8999-999999999999': [12]}

Member next input
The prompt template is the same as the prompt for 'Member next input', while the content of #Document is the merge of the document from two members.
First, merge the documents of member 1 and member 9 to obtain the new result.

Member 1 and Member 9 merge: f1ec3dd8-b6e7-4af5-a589-fda0b7693f93

Since the merged output is identical to the original output of member 1 but differs from the original output of member 9, member 9 has been removed. Next, merge the documents of member 1 and member 12 to obtain the new result.

Member 1 and Member 12 merge: f1ec3dd8-b6e7-4af5-a589-fda0b7693f93

Remove member 12 because the merged output is identical to the original output of member 1 and differs from the original output of member 12. At this point, all members have only one unique result. The leader will provide the final answer based on this result.

Here are the responses from all the members. Each member sees different segments of a document, and these segments do not intersect with each other. The correct answer may appear in any one or several members' responses.
Note that if a minority of members find information relevant to the question while the majority reply that the document does not contain information relevant to the question, you should pay attention to the replies from those members who found relevant information.
 # Member Response:
Member 1: 121,277

 # Task Description:
Answer the question based on the given passages. Only give me the answer and do not output any other words.

 # Task Objective:
Did Leshan or Quanzhou have a population of 658,798 in 2002?

 # Determination:
Based on the above information, you need to determine if you can solve the task objective. You have two choices:
1. If members' responses cannot solve the task objective, or if their responses contain conflicting answers, provide a new instruction for them to answer again.
2. Else, if the task objective can be solved, give your final answer as concisely as you can, using a single phrase if possible. Do not provide any explanation.
Your output must following the JSON format: {{"type": "answer", "content": "your_answer_content"}} or {{"type": "instruction", "content": "your_instruction_content"]}}

Table 8: Prompt template for Leader third input on the Multi-document Question Answering Task.

 # Document:
`{member_document}`

 # Instruction:
What was the population of Leshan in 2002?

You are an experienced writer; please summarize the content in the document related to the instructions in a <scratchpad> tag, then describe your response." Your output must following the JSON format: {{"type": "response", "content": "your_response_content"}}
The "content" needs to be as concise as possible.

Table 9: Prompt template for Member second input on the Multi-document Question Answering Task.

 # Document:
`{member_document}`

 # Instruction:
What was the population of Quanzhou in 2002?

You are an experienced writer; please summarize the content in the document related to the instructions in a <scratchpad> tag, then describe your response." Your output must following the JSON format: {{"type": "response", "content": "your_response_content"}}
The "content" needs to be as concise as possible.

Table 10: Prompt template for Member third input on the Multi-document Question Answering Task.

Leader last input
leader summarizes the final answer based on previous information. Details are shown in Table 14.

Leader last output
"type": "answer", "content": "f1ec3dd8-b6e7-4af5-a589-fda0b7693f93"

Here are the responses from all the members. Each member sees different segments of a document, and these segments do not intersect with each other. The correct answer may appear in any one or several members' responses.
Note that if a minority of members find information relevant to the question while the majority reply that the document does not contain information relevant to the question, you should pay attention to the replies from those members who found relevant information.
 # Member Response:
Member 1: 658,798

 # Task Description:
Answer the question based on the given passages. Only give me the answer and do not output any other words.

 # Task Objective:
Did Leshan or Quanzhou have a population of 658,798 in 2002?

 # Determination:
Based on the above information, you need to determine if you can solve the task objective. You have two choices:
1. If members' responses cannot solve the task objective, or if their responses contain conflicting answers, provide a new instruction for them to answer again.
2. Else, if the task objective can be solved, give your final answer as concisely as you can, using a single phrase if possible. Do not provide any explanation.
Your output must following the JSON format: {{"type": "answer", "content": "your_answer_content"}} or {{"type": "instruction", "content": "your_instruction_content"]}}

Table 11: Prompt template for Leader last input on the Multi-document Question Answering Task.

B.4 Math.Find

B.4.1 Find the largest number

Leader first input
First, the leader recruits members according to the task description as shown in Table 2.
Leader first output
{"type": "member", "content": "Math member"}
Leader next input
The prompt for the Leader next input is shown in Tabel 15.
The content of #Task Description and #Task Objective is derived from user input.

Leader first output
{"type": "instruction", "content": "You should answer with only one number, no other words. The largest number of the list is: "}
Member first input
The prompt for the Member first input is shown in Tabel 16.
The content of #Document is a part of the complete context, with variations for each member.
The content of #Instruction originates from the first output of the leader.

Member first output
Each member returns the maximum value of the numbers in their documents. The results of each member are recorded and passed to the leader.
Leader last input
The prompt for the Leader last input is shown in Tabel 17.
Leader finds the maximum value for the entire document based on the outputs of all members.

Leader last output
{"type": "answer", "content": "94"}

B.4.2 Find the second largest number

For other tasks in Math.Find, the prompt template remains the same; it will be omitted from here on.
Leader first output
{"type": "instruction", "content": "You should answer with only one number, no other words. The largest number and second-largest number of the

You are the leader of a team of {member_nums} members. Your team will need to collaborate to solve a task. The rule is:
1. Only you know the task description and task objective; the other members do not.
2. But they will receive different documents that may contain answers, and you need to send them an instruction to query their document.
3. Your instruction need to include your understanding of the task and what you need them to focus on. If necessary, your instructions can explicitly include the task objective.
4. Finally, you need to complete the task based on the query results they return.
 # Task Description:
Finding the corresponding value from a dictionary and a key.

 # Task Objective:
Key: "2b114db0-d87e-42d2-9b4c-0b1f115976ad"
The value associated with the specified key is:

 # Generate Instruction for Members:
Now, you need to generate an instruction for all team members. You can ask them to answer a certain question, or to extract information related to the task, based on their respective documents.
Your output must following the JSON format: {{"type": "instruction", "content": "your_instruction_content"}}

Table 12: Prompt template for Leader next input on the Retrieve.KV Task.

list are: "}

Member first output

Member 1: 81, 80	*Member 2: 81, 80*
Member 3: 81, 80	*Member 4: 79, 78*
Member 5: 81, 80	*Member 6: 81, 80*
Member 7: 81, 80	*Member 8: 81, 80*
Member 9: 81, 80	*Member 10: 81, 80*
Member 11: 81, 80	*Member 12: 80, 79*
Member 13: 81, 80	*Member 14: 81, 80*
Member 15: 81, 80	*Member 16: 79, 78*
Member 17: 81, 80	*Member 18: 81, 80*
Member 19: 81, 80	*Member 20: 81, 80*
Member 21: 81, 80	*Member 22: 81, 80*
Member 23: 81, 80	*Member 24: 81, 80*
Member 25: 81, 80	*Member 26: 81, 80*
Member 27: 81, 80	*Member 28: 81, 80*
Member 29: 81, 80	*Member 30: 81, 80*

 # Document:
{member_document}

 # Instruction:
Finding the corresponding value from a dictionary and a key.
Key: "2b114db0-d87e-42d2-9b4c-0b1f115976ad"
The value associated with the specified key is:

You are an experienced writer; please summarize the content in the document related to the instructions in a <scratchpad> tag, then describe your response." Your output must following the JSON format: {{"type": "response", "content": "your_response_content"}}
The "content" needs to be as concise as possible.

Table 13: Prompt template for Member first input on the Retrieve.KV Task.

Member first output
"type": "answer", "content": "80"

B.4.3 Find the third largest number

Leader first output
{"type": "instruction", "content": "You should answer with only one number, no other words. The largest number, second-largest number and third-largest number of the list are:"}

Member first output

Member 1: 86, 85, 84	*Member 2: 86, 85, 84*
Member 3: 86, 85, 84	*Member 4: 87, 86, 85*
Member 5: 86, 85, 84	*Member 6: 86, 85, 84*
Member 7: 86, 85, 84	*Member 8: 86, 85, 84*
Member 9: 87, 86, 85	*Member 10: 87, 86, 85*
Member 11: 86, 85, 84	*Member 12: 86, 85, 84*
Member 13: 86, 85, 84	*Member 14: 88, 87, 86*
Member 15: 86, 85, 84	*Member 16: 87, 86, 85*
Member 17: 86, 85, 84	*Member 18: 86, 85, 84*
Member 19: 86, 85, 84	*Member 20: 86, 85, 84*
Member 21: 86, 85, 84	*Member 22: 86, 85, 84*
Member 23: 86, 85, 84	*Member 24: 86, 85, 84*
Member 25: 86, 85, 84	*Member 26: 88, 87, 86*
Member 27: 86, 85, 84	*Member 28: 86, 85, 84*
Member 29: 86, 85, 84	*Member 30: 86, 85, 84*

Member first output
{"type": "answer", "content": "80"}
The method for finding the minimum value is similar to that for finding the maximum value, and it will not be further elaborated here.

Here are the responses from all the members. Each member sees different segments of a document, and these segments do not intersect with each other. The correct answer may appear in any one or several members' responses.
Note that if a minority of members find information relevant to the question while the majority reply that the document does not contain information relevant to the question, you should pay attention to the replies from those members who found relevant information.
 # Member Response:
Member 1: f1ec3dd8-b6e7-4af5-a589-fda0b7693f93

 # Task Description:
Finding the corresponding value from a dictionary and a key.

 # Task Objective:
Key: "2b114db0-d87e-42d2-9b4c-0b1f115976ad"
The value associated with the specified key is:

 # Determination:
Based on the above information, you need to determine if you can solve the task objective. You have two choices:
1. If members' responses cannot solve the task objective, or if their responses contain conflicting answers, provide a new instruction for them to answer again.
2. Else, if the task objective can be solved, give your final answer as concisely as you can, using a single phrase if possible. Do not provide any explanation.
Your output must following the JSON format: {{"type": "answer", "content": "your_answer_content"}} or {{"type": "instruction", "content": "your_instruction_content"]}}

Table 14: Prompt template for Leader last input on the Retrieve.KV Task.

C Additional Results

This section complements the experimental results mentioned in the main body of this paper.

You are the leader of a team of {member_nums} members. Your team will need to collaborate to solve a task. The rule is:
1. Only you know the task description and task objective; the other members do not.
2. But they will receive different documents that may contain answers, and you need to send them an instruction to query their document.
3. Your instruction need to include your understanding of the task and what you need them to focus on. If necessary, your instructions can explicitly include the task objective.
4. Finally, you need to complete the task based on the query results they return.
 # Task Description:
Finding special integers in a lengthy list.

 # Task Objective:
You should answer with only one number, no other words. The largest number of the list is:

 # Generate Instruction for Members:
Now, you need to generate an instruction for all team members. You can ask them to answer a certain question, or to extract information related to the task, based on their respective documents.
Your output must following the JSON format: {{"type": "instruction", "content": "your_instruction_content"}}

Table 15: Prompt template for Leader next input on the Math.Find Task.

 # Document:
{member_document}

 # Instruction:
You should answer with only one number, no other words. The largest number of the list is:

You are an experienced writer; please summarize the content in the document related to the instructions in a <scratchpad> tag, then describe your response." Your output must following the JSON format: {{"type": "response", "content": "your_response_content"}}
The "content" needs to be as concise as possible.

Table 16: Prompt template for Member first input on the Math.Find Task.

Here are the responses from all the members. Each member sees different segments of a document, and these segments do not intersect with each other. The correct answer may appear in any one or several members' responses.
Note that if a minority of members find information relevant to the question while the majority reply that the document does not contain information relevant to the question, you should pay attention to the replies from those members who found relevant information.
 # Member Response:
Member 1: 94 Member 2: 94 Member 3: 94 Member 4: 94 Member 5: 94 Member 6: 94 Member 7: 94 Member 8: 94 Member 9: 94 Member 10: 94 Member 11: 94 Member 12: 94 Member 13: 94 Member 14: 94 Member 15: 94 Member 16: 94 Member 17: 94 Member 18: 94 Member 19: 94 Member 20: 94 Member 21: 94 Member 22: 94 Member 23: 94 Member 24: 94 Member 25: 93 Member 26: 94 Member 27: 94 Member 28: 94 Member 29: 94 Member 30: 94

 # Task Description:
Finding special integers in a lengthy list.

 # Task Objective:
You should answer with only one number, no other words. The largest number of the list is:

 # Determination:
Based on the above information, you need to determine if you can solve the task objective. You have two choices:
1. If members' responses cannot solve the task objective, or if their responses contain conflicting answers, provide a new instruction for them to answer again.
2. Else, if the task objective can be solved, give your final answer as concisely as you can, using a single phrase if possible. Do not provide any explanation.
Your output must following the JSON format: {{"type": "answer", "content": "your_answer_content"}} or {{"type": "instruction", "content": "your_instruction_content"]}}

Table 17: Prompt template for Leader last input on the Math.Find Task.

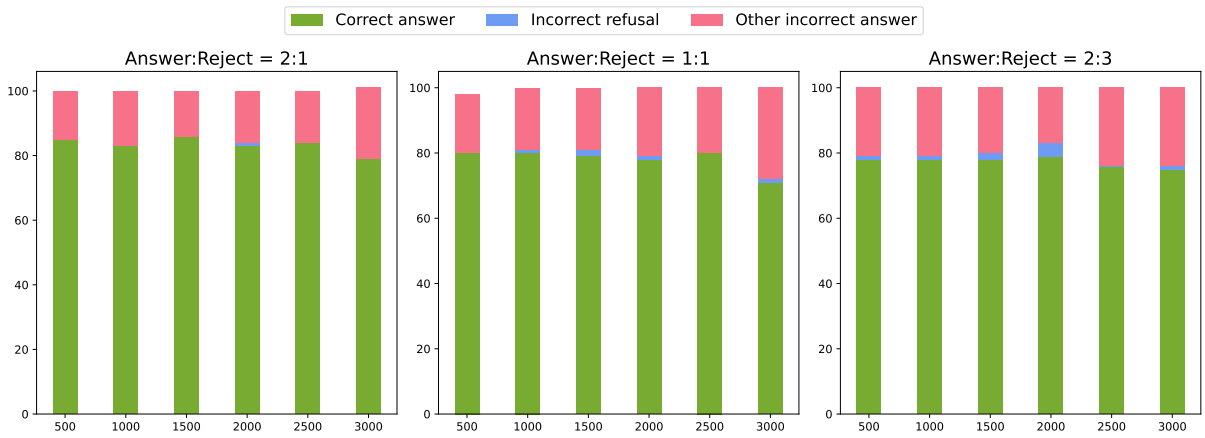

Figure 8: Effect 'Reject' type data on model QA capability. 'Answer' and 'Reject' represent two types of data. For the former, the documents contain answers to questions; whereas for the latter, they do not.